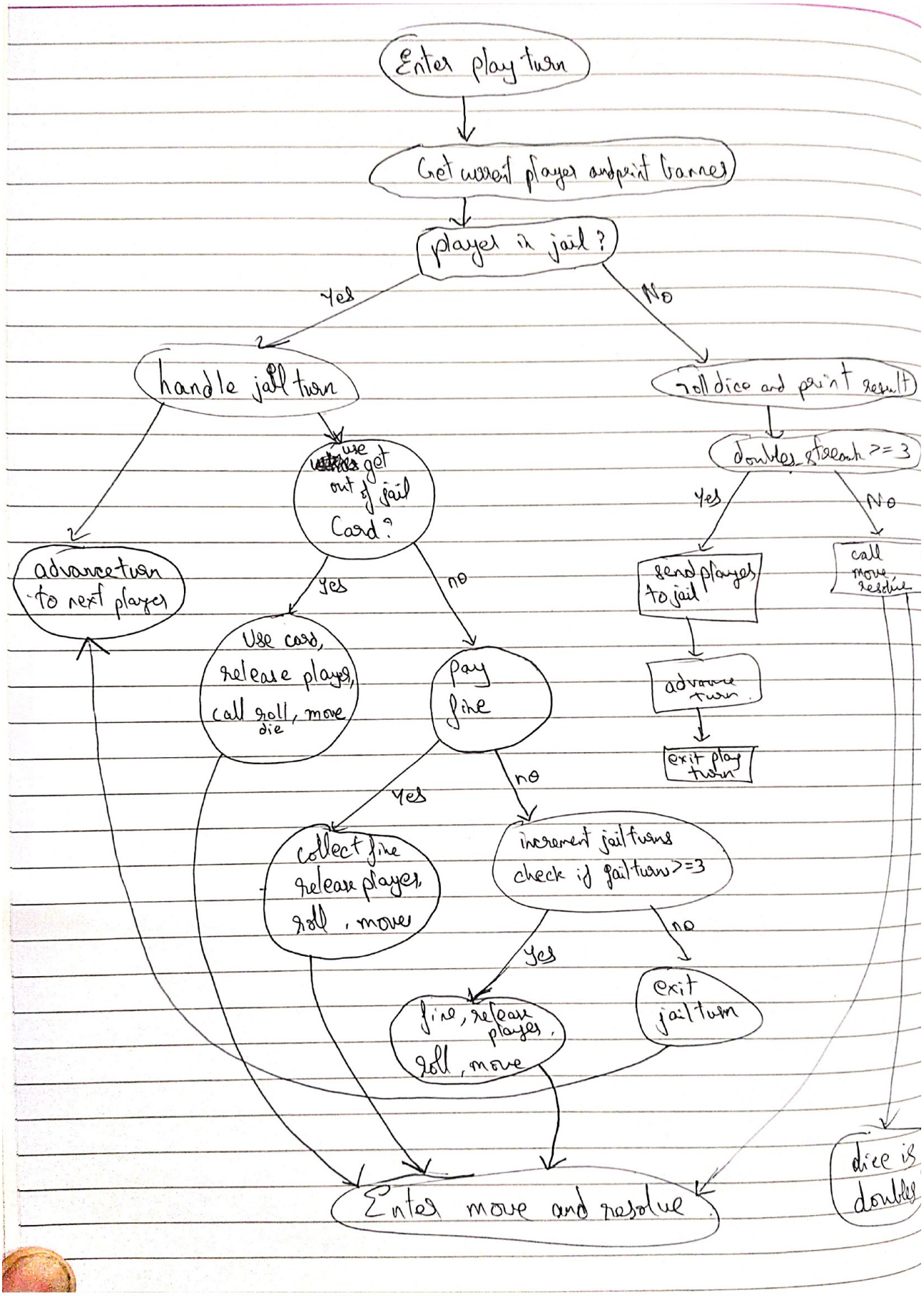
White Box Report

1.1 Control Flow Graph

For the control flow graph, I modeled the principal execution path of the MoneyPoly program. The graph starts from `main()`, where player names are collected and the `Game` object is created, and then follows the main gameplay flow through `Game.run()`, `play_turn()`, and `_move_and_resolve()`. I expanded `_handle_jail_turn()`, `_apply_card()`, and `_handle_property_tile()` because these functions contain the most important decision points, branch paths, and state changes in the program. To keep the handwritten CFG readable, sequential statements were grouped into related blocks and repeated post-branch flows were merged into common nodes while preserving the actual control structure of the program. The attached hand-drawn diagram was created from the actual code and includes clearly labeled nodes and arrows to show the flow of control.



Enter move and Resolve

Move, Compute position, print tile

1.2 Code Quality Analysis

For this part, I used `pylint` to analyze the copied MoneyPoly code placed under `whitebox/code/moneypoly/`. I performed the cleanup iteratively and recorded each atomic step as a separate commit, as required in the assignment. The goal was to improve code quality without changing the intended gameplay behavior unless a refactor was necessary for maintainability.

Tool Used

```
PYTHONPATH='whitebox/code/moneypoly' .venv/bin/pylint whitebox/code/moneypoly/main.py
whitebox/code/moneypoly/moneypoly/*.py
```

Iterative Changes

1. `Iteration 1: Add MoneyPoly code to whitebox working tree`
2. Copied the MoneyPoly source into `whitebox/code/moneypoly/` so the white-box deliverable matches the required assignment structure.
3. `Iteration 2: Remove unused imports and tidy lint hygiene`
4. Removed unused imports from `bank.py`, `dice.py`, `game.py`, and `player.py`.
5. Removed one unused local variable from `player.py`.
6. Fixed missing final newlines.
7. `Iteration 3: Fix direct pylint style and exception warnings`
8. Replaced a singleton comparison in `board.py`.
9. Replaced a bare `except` in `ui.py` with `except ValueError`.
10. Removed unnecessary `else` structure in `property.py`.
11. Fixed several direct style warnings in `game.py`.
12. Defined `doubles_streak` inside `Dice.__init__()`.
13. `Iteration 4: Add missing module and function docstrings`
14. Added missing module docstrings across the package.
15. Added missing function docstrings in `main.py`.
16. Added missing class docstrings where needed.
17. `Iteration 5: Reformat card definitions for pylint compliance`
18. Reformatted the Chance and Community Chest card data in `cards.py` to remove line-length violations while preserving the same data.
19. `Iteration 6: Refactor card handling to reduce branch complexity`
20. Refactored `Game._apply_card()` into smaller helper methods with dispatch-based handling.

21. This reduced branching complexity and made the card logic easier to understand and maintain.

22. `Iteration 7: Resolve remaining pylint structural warnings`

23. Added targeted pylint suppressions for domain-model-heavy classes and constructor signatures where the warning reflected the shape of the game model rather than a correctness problem.
24. This was applied only to the remaining structural cases after direct cleanup and refactoring had already been completed.

Final Result

After all iterations, the final pylint run reported:

```
Your code has been rated at 10.00/10
```

Summary

The code quality analysis improved the MoneyPoly codebase through multiple small commits instead of one large cleanup. The major improvements included removal of unused code, better exception handling, improved documentation, cleaner formatting, and a refactor of card-processing logic. The final white-box working copy under `whitebox/code/moneypoly/` achieved a pylint score of `10.00/10`.

1.3 White Box Test Cases

For the white-box testing section, I designed tests directly from the control flow of the main gameplay logic. The goal was to cover important decisions, key state changes, and relevant edge cases rather than only testing surface-level outputs. I wrote the tests under `whitebox/tests/` and followed an iterative red-green-fix approach: first write a focused failing test, then identify the root cause, then apply the smallest code fix needed.

Why These Tests Were Needed

The MoneyPoly code contains several decision-heavy paths: - dice rolling and doubles handling - winner selection - movement across the board and passing Go - property purchases, rent, mortgages, and trades - jail behavior - card-deck actions - bankruptcy and cleanup

These areas were chosen because they directly influence game state and can easily hide logical mistakes. The tests were also aligned with the CFG created in Section 1.1, especially the branches inside `play_turn()`, `_move_and_resolve()`, `_handle_jail_turn()`, `_apply_card()`, and `_handle_property_tile()`.

Errors Found And Fixed

1. `Error 1: Fix dice roll range and add white-box test`
2. Problem found: dice rolls used `randint(1, 5)` instead of a six-sided range.

3. Why the test was needed: dice values control movement and doubles logic, so the wrong range affects many later branches.

4. Fix: changed the dice roll range to `1..6`.

5. `Error 2: Fix winner selection to use highest net worth`

6. Problem found: `find_winner()` returned the player with minimum net worth.
7. Why the test was needed: final winner calculation is a core outcome of the program.

8. Fix: changed winner selection from `min(...)` to `max(...)`.

9. `Error 3: Award Go salary when passing the board start`

10. Problem found: Go salary was only awarded when landing exactly on position `0`, not when passing it.
11. Why the test was needed: player movement across the board is a key state transition and passing Go is an expected edge case.

12. Fix: updated movement logic to detect wrap-around and award salary.

13. `Error 4: Allow property purchase with exact balance`

14. Problem found: players could not buy a property if their balance exactly matched the price.
15. Why the test was needed: exact-boundary values are explicitly required edge cases in the assignment.

16. Fix: changed affordability check from `<=` to `<`.

17. `Error 5: Credit rent payments to the property owner`

18. Problem found: rent was deducted from the tenant but not credited to the owner.
19. Why the test was needed: this is a direct money-transfer path and an important integration between players.

20. Fix: added the owner balance update in `pay_rent()`.

21. `Error 6: Require full group ownership before doubling rent`

22. Problem found: rent was doubled when the owner had only one property in the group.
23. Why the test was needed: rent calculation depends on key variable state, especially group ownership.

24. Fix: changed the ownership check from `any(...)` to `all(...)`.

25. `Error 7: Deduct player balance when paying jail fine`

26. Problem found: voluntarily paying the jail fine released the player but did not deduct money from the player.
27. Why the test was needed: jail handling contains several important branches with financial effects.

28. Fix: deducted the fine before releasing the player.

29. `Error 8: Guard empty card decks against zero-division`

30. Problem found: `cards_remaining()` and `__repr__()` failed for an empty deck.
31. Why the test was needed: empty-container edge cases are relevant white-box test scenarios.

32. Fix: added explicit empty-deck guards.

33. `Error 9: Transfer cash to seller during property trades`

34. Problem found: the buyer paid cash, ownership changed, but the seller did not receive the money.
35. Why the test was needed: property trade is a multi-state transaction and needed direct money-flow verification.

36. Fix: credited the seller during successful trades.

37. `Error 10: Preserve mortgage state on failed unmortgage`

38. Problem found: if the player could not afford the unmortgage cost, the property still became unmortgaged.
39. Why the test was needed: this is an order-of-operations bug caused by state mutation before validation.

40. Fix: delayed the state change until after the affordability check.

41. `Error 11: Reduce bank funds when issuing loans`

42. Problem found: loans increased the player balance but did not reduce bank funds.
43. Why the test was needed: the bank and player states should stay consistent after loan issuance.

44. Fix: debited bank funds when the loan was issued.

45. `Error 12: Enforce minimum player count in game setup`

46. Problem found: the program prompt required at least two players, but `Game()` still accepted zero or one player.
47. Why the test was needed: setup validation is part of the documented program contract and `main()` was already prepared to surface setup errors.

48. Fix: added a `ValueError` guard in `Game.__init__()` when fewer than two player names are provided.

49. `Error 13: Preserve turn order after player elimination`

50. Problem found: if the current player went bankrupt during their turn, `play_turn()` still advanced the index and could skip the next active player or print a doubles retry message for an eliminated player.
51. Why the test was needed: bankruptcy cleanup changes the player list during active control flow, which makes turn-order handling a high-risk white-box branch.

52. Fix: corrected `_check_bankruptcy()` index handling and short-circuited `play_turn()` when the active player is eliminated mid-turn.

53. `Error 14: Include owned assets in net worth`

54. Problem found: `Player.net_worth()` returned only cash balance and ignored owned properties, which made standings and winner selection wrong in asset-heavy states.
55. Why the test was needed: `net_worth()` is used by both the UI and final winner logic, so incorrect asset valuation affects multiple program outcomes.

56. Fix: updated `net_worth()` to include owned property value, counting mortgaged properties at mortgage value.

57. `Error 15: Expose pre-roll actions in the main turn flow`

58. Problem found: the interactive mortgage / trade / loan menu existed but was never called from `play_turn()`, so those features were unreachable in normal gameplay.
59. Why the test was needed: white-box analysis revealed dead control-flow branches that were implemented but not connected to the actual game loop.

60. Fix: inserted the pre-roll menu call into the normal non-jail turn path before the dice roll.

61. `Error 16: Use real bank payouts for mortgages`

62. Problem found: mortgage payouts were implemented as `bank.collect(-amount)`, which reduced bank reserves but also corrupted the bank's `total_collected` accounting.
63. Why the test was needed: mortgage handling is a money-transfer path and should preserve internal bank accounting consistency.

64. Fix: changed mortgage payouts to use `Bank.pay_out()` instead of a fake negative collection.

65. `Error 17: Prevent emergency loans from overdrawing the bank`

66. Problem found: emergency loans could exceed bank reserves and drive the bank balance negative.
67. Why the test was needed: the assignment expects explicit handling of invalid states, and silent bank overdrafts are a direct financial-model bug.

68. Fix: routed loan issuance through `Bank.pay_out()` so oversized loans fail fast with `ValueError` and no state changes.

69. `Error 18: Keep mortgage transactions atomic on bank failure`

70. Problem found: mortgage processing set `prop.is_mortgaged = True` before confirming the bank could fund the payout, which left a half-applied transaction when `Bank.pay_out()` failed.
71. Why the test was needed: money-transfer operations should be atomic, especially when a failed payout would otherwise mutate game state without compensating the player.

72. Fix: moved the bank payout check ahead of the mortgage-state mutation so the property remains unchanged if the bank cannot pay.

73. `Error 19: Reject negative bank collections`

74. Problem found: `Bank.collect(-amount)` reduced both reserves and `total_collected`, which contradicted the method contract and silently corrupted bank accounting.
75. Why the test was needed: negative collection is an invalid financial input and should not silently mutate internal state.

76. Fix: changed `Bank.collect()` to fail fast with `ValueError` for negative amounts.

77. `Error 20: Exclude railroads from special-tile helper`

78. Problem found: `Board.is_special_tile()` returned `True` for railroad squares even though the helper's docstring described only non-property special tiles.
79. Why the test was needed: helper semantics matter for white-box reasoning and should match the documented meaning of the method.
80. Fix: narrowed `is_special_tile()` so it excludes railroads while still returning `True` for real special tiles such as chance, jail, and taxes.

Additional Branch Coverage Added

After fixing the detected errors, I added more white-box tests to cover important branches that already behaved correctly: - card action branches - property tile decision branches - jail-turn branches - special tile resolution branches - mortgage, auction, and bankruptcy branches - play-turn doubles and triple-doubles branches - board, player, bank, and card-deck helper behavior - failure paths where properties or tiles resolve to missing objects

These additional tests were committed incrementally as `Test 1` through `Test 15`, so the final suite reflects both bug-finding tests and branch-expansion tests.

These tests improved confidence that the corrected code now behaves properly across the main gameplay paths.

Verification

I verified the final white-box test suite with:

```
PYTHONPATH='whitebox/code/moneypoly' .venv/bin/pytest whitebox/tests -q
```

Final result:

```
84 passed in 0.05s
```

Summary

The white-box testing process found multiple logical issues in movement, winner selection, setup validation, turn-order handling after bankruptcy, property transactions, helper semantics, jail handling, card-deck edge cases, bank accounting, and trade flow. Each

detected error was fixed through a separate small commit, and the final suite now covers a broad set of branches, variable states, helper behaviors, and edge cases derived from the control flow graph. The final submission now includes `Error 1` through `Error 20` commits for defect correction and `Test 1` through `Test 15` commits for branch and edge-case expansion.